



PROGETTO **TYPESCRIPT** - MASTER “*FULL STACK DEVELOPMENT E AI*”
DI START2IMPACT

MOOVE: URBAN MICROMOBILITY SYSTEM IN TYPESCRIPT

REPO GITHUB: [HTTPS://GITHUB.COM/SADSOTTI/MOOVE-TS](https://github.com/sadsotti/moove-ts)

CODEPEN LIVE: [HTTPS://CODEPEN.IO/SADSOTTI/PEN/GBARDBJ](https://codepen.io/sadsotti/pen/gbardbj)

PRESENTATO DA: LORENZO SOTTILE



OBIETTIVO E ENTITÀ

Obiettivo del Progetto:

- Simulare un sistema di micromobilità urbana condivisa (es. bici, scooter, monopattini) con logiche di prenotazione e gestione stato.
- Dimostrare l'applicazione concreta di TypeScript per un codice robusto e scalabile.
- Consolidare i principi di Programmazione Orientata agli Oggetti (OOP) e il Design a Interfacce.

Le Entità Fondamentali:

- **IVehicle / Vehicle** (Mezzo): Rappresenta il veicolo (es. 'bike', 'scooter', 'kick scooter'), con id e status (available, in_use, maintenance). Include il metodo assignUser().
- **IUser / User** (Utente): Rappresenta user, con firstName, lastName, email, preferredPaymentMethod. Include il metodo bookVehicle().
- **ICity / City** (Città): Gestisce i availableVehicles in una città specifica (es. "Milan"). Permette di addVehicle() e showVehicleStatus().



LE MIE SCELTE: TYPESCRIPT E INTERFACCE

Perché **TypeScript**?

- Tipizzazione Forte: Cattura errori in fase di sviluppo (es. tipi non corrispondenti), aumentando la stabilità e riducendo i bug.
- Chiarezza del Codice: Migliora la leggibilità e l'autocompletamento dell'IDE, rendendo il codice più comprensibile e facile da mantenere.
- Unioni di Tipi ('bike' | 'scooter'): Garantiscono che solo valori validi siano assegnati, migliorando la sicurezza dei dati.

Perché le **Interfacce** (IVehicle, IUser, ICity)?

- Contratti Chiari: Definiscono esplicitamente cosa un'entità "può fare" o "deve avere", migliorando il design e la scalabilità.
- Disaccoppiamento: Favoriscono un'architettura più flessibile e modulare, dove i componenti dipendono da astrazioni piuttosto che da implementazioni concrete.



FUNZIONALITÀ E SIMULAZIONE PRATICA

Logica del Flusso (Esempi dal Codice):

1. Creazione: Si istanziano `Vehicle('bike', 'B001')`, `User('Alice', ...)` e `City('Milan')`.
2. Aggiunta: `milan.addVehicle(bike1)` registra il mezzo nella città.
3. Prenotazione: `userAlice.bookVehicle(bike1)` verifica la disponibilità. Se disponibile, il mezzo chiama `bike1.assignUser(userAlice)`.
4. Gestione Status: Lo stato del mezzo passa da `available` a `in_use`.
5. Casi Limite: Il sistema gestisce tentativi di prenotazione di mezzi già `in_use` o `in maintenance`, mostrando messaggi di errore/warning.

Output in Console (Simulazione):

- Il codice produce un output dettagliato che traccia ogni operazione e il cambio di stato dei mezzi.
 - "[Vehicle Created] bike ID: B001, Status: available"
 - "[Booking in Progress] Alice Rossi is trying to book the bike ID:B001."
 - "[Vehicle Assigned] The bike ID:B001 has been assigned to Alice Rossi."
 - "Vehicle Status in Milan: - bike (ID: B001): Status -> in use"

CONCLUSIONI E FUTURO

Il progetto ha consolidato la mia comprensione dell'OOP, evidenziando la potenza della tipizzazione statica di **TypeScript** e l'importanza del design modulare tramite interfacce per codice flessibile e manutenibile.

Per il futuro, vedo diverse evoluzioni: l'integrazione di un sistema di pagamento e di punti di ritiro/ricarica, la persistenza dei dati tramite **backend** e **database**, e lo sviluppo di una UI per un'esperienza più completa.

IL MIO GITHUB: [HTTPS://GITHUB.COM/SADSOTTI](https://github.com/sadsotti)

REPO GITHUB: [HTTPS://GITHUB.COM/SADSOTTI/MOOVE-TS](https://github.com/sadsotti/moove-ts)

IL MIO CODEPEN: [HTTPS://CODEPEN.IO/SADSOTTI](https://codepen.io/sadsotti)

CODEPEN LIVE: [HTTPS://CODEPEN.IO/SADSOTTI/PEN/GBARDBJ](https://codepen.io/sadsotti/pen/gbardbj)

PRESENTATO DA: **LORENZO SOTTILE**



PROGETTO **TYPESCRIPT** - MASTER “*FULL STACK DEVELOPMENT E AI*”
DI START2IMPACT

grazie